SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

ASSESSMENT TOOL 1

Analytical Problem Solving

Course Name: Cloud Computing and Big Data Analytics
Course Code: CSA15
Course Outcome Covered: CO2 (BL3)

Name	**O. Eshanth Reddy**
Register No.	**1924260199**
Course Code	CSA15
Assessment Weightage	40%
Total Marks	50 (5 questions)
Date	26-10-2026

Submitted by: O. Eshanth Reddy (1924260199)

Code of Conduct

I **O. Eshanth Reddy** (Reg. No. **1924260199**) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Contents

Question 1: Type-1 vs Type-2 hypervisors for mission-critical cloud workloads

Question 2: Workload isolation for finance, healthcare and student portals

Question 3: VM host utilization snapshot: bottlenecks and corrective actions

Question 4: Software Defined Datacenter vs traditional datacenter

Question 5: Identity mismatch in a multi-cloud federation: secure identity and privacy design

Overall conclusion and references

Rubric Coverage

Every answer follows the same structure so that the rubric criteria are clearly visible: **problem interpretation**, **analytical reasoning** (cause and effect), **method selection** (appropriate virtualization and security concepts), **solution quality** (feasible, technically sound recommendations) and **presentation** (headings, tables and diagrams).

Question 1: Type-1 vs Type-2 Hypervisors for Mission-Critical Cloud Workloads

1.1 Problem Interpretation

A cloud datacenter that hosts mission-critical applications (for example banking, healthcare or e-governance systems) needs three things from its virtualization layer: **high and predictable performance**, **strong security**, and **manageability at scale**. The task is to compare Type-1 and Type-2 hypervisors against these three criteria and to justify which one is the better choice. The comparison is therefore not about which product is "better" in general, but which architecture fits an environment where downtime and data leakage are very costly.

1.2 Architecture of the Two Hypervisor Types

A **hypervisor** (virtual machine monitor) is the software layer that creates and runs virtual machines (VMs) by sharing the physical CPU, memory, storage and network among them. The difference between the two types lies in what sits directly beneath the hypervisor.

- **Type-1 (bare-metal) hypervisor:** installed directly on the physical server. It controls the hardware itself and schedules the VMs. Examples: VMware ESXi, Microsoft Hyper-V, Xen and KVM (KVM turns the Linux kernel into a Type-1 style hypervisor).
- **Type-2 (hosted) hypervisor:** installed as an application on top of a general-purpose host operating system such as Windows, Linux or macOS. Every hardware request from a VM passes through the host OS. Examples: Oracle VirtualBox and VMware Workstation.

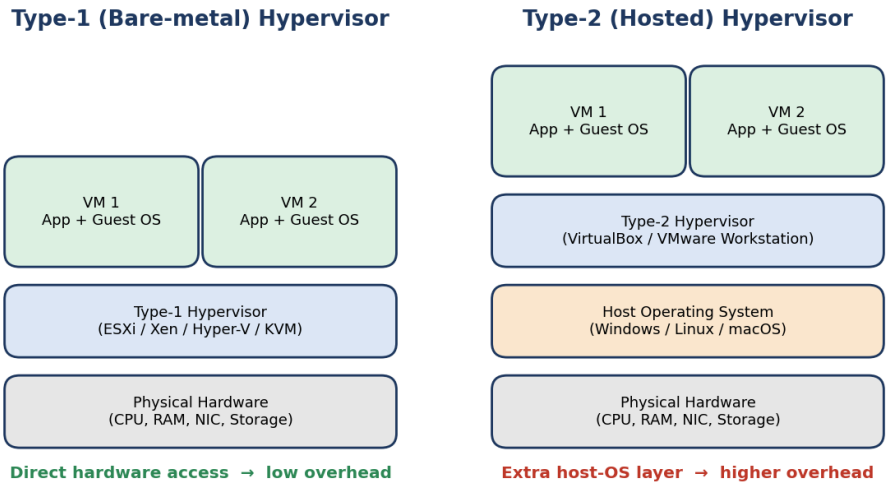


Figure 1.1: Layered architecture of Type-1 and Type-2 hypervisors

1.3 Comparative Analysis

Criterion	Type-1 (Bare-metal)	Type-2 (Hosted)
Position in stack	Runs directly on hardware	Runs on top of a host operating system
Performance	Near-native; no extra OS layer; direct control of CPU scheduling, memory and I/O	Added overhead from the host OS; VM performance depends on other host applications
Security	Small, purpose-built code base (small attack surface); no general-purpose applications on the host	Large attack surface: host OS, browsers, drivers and user applications can all be attacked
Manageability	Central management (e.g. vCenter, System Center), live migration, HA clusters, automated patching	Managed per machine; no native clustering or live migration at datacenter scale
Reliability	Host-level fault tolerance and failover are supported	A host OS crash or reboot takes every VM down
Scalability	Hundreds of VMs per host and thousands per cluster	Suitable for a few VMs on a desktop
Typical use	Production datacenters and public/private clouds	Testing, development, training and desktop labs
Setup effort	Needs dedicated, certified hardware and skilled admins	Very easy; installs like any application

Performance analysis

In a Type-1 design the hypervisor talks to the hardware directly, using hardware-assisted virtualization (Intel VT-x / AMD-V) and paravirtualized drivers. A VM's disk or network request therefore needs one translation step only. In a Type-2 design the same request travels from the guest OS, to the hypervisor, to the host OS and its drivers, and only then to the hardware. This additional path adds latency and makes performance **unpredictable**, because the host OS may be running antivirus scans, updates or user applications at the same time. For mission-critical applications that must meet strict response-time targets, this variability is unacceptable.

Security analysis

Security depends strongly on the size of the trusted computing base (TCB), that is, the amount of code that must be correct for the system to be safe. A Type-1 hypervisor has a compact TCB and is usually hardened, signed and updated by the vendor. A Type-2 hypervisor inherits the entire host OS in its TCB: any vulnerability in the host kernel, a driver or a browser can let an attacker take control of every VM. Type-1 platforms additionally support secure boot, TPM-based attestation, encrypted VMs and fine-grained role-based access control. This does not make Type-1 immune (hypervisor vulnerabilities do exist), but its attack surface is much smaller and easier to defend.

Manageability analysis

A cloud datacenter has hundreds or thousands of hosts. Type-1 platforms provide cluster-wide management: **live migration** of running VMs between hosts, **distributed resource scheduling** to balance load, **high availability** that restarts VMs automatically after a host failure, and API-based automation for orchestration tools. Type-2 hypervisors lack these functions because each host is managed individually, which makes patching, monitoring and capacity management slow and error-prone.

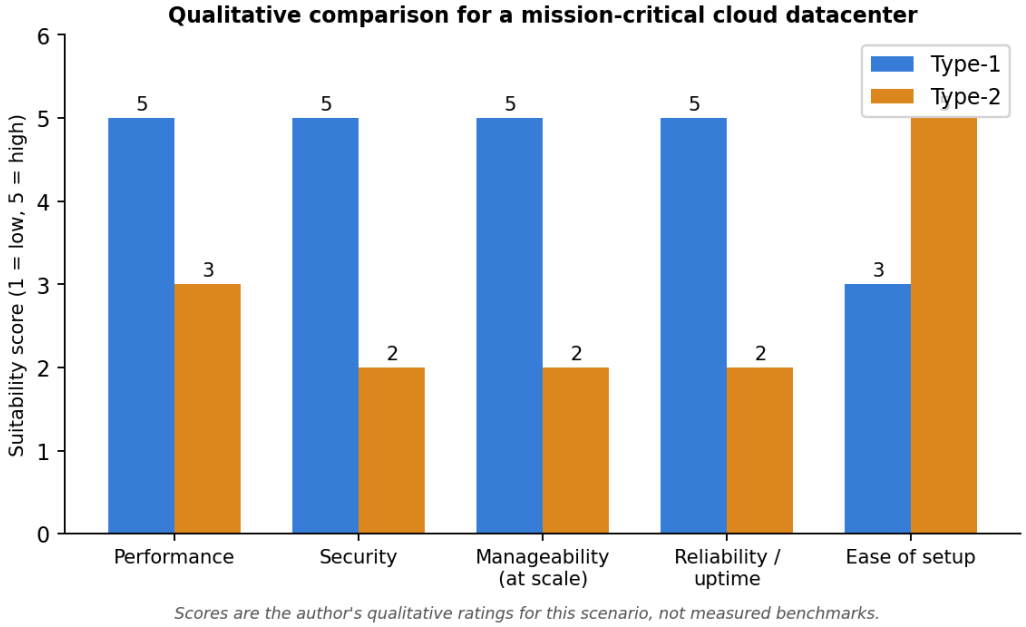


Figure 1.2: Qualitative suitability comparison for a mission-critical datacenter

1.4 Justification of the Better Choice

A Type-1 (bare-metal) hypervisor is the better choice for a cloud datacenter hosting mission-critical applications. The decision follows directly from the analysis above:

- **Performance:** direct hardware access gives near-native, predictable speed and lets the provider guarantee service-level agreements (SLAs).
- **Security:** a smaller attack surface, hardware-rooted trust and strong isolation between tenants reduce the risk of data breaches.
- **Manageability:** clustering, live migration, high availability and automation allow maintenance without downtime and efficient use of hardware.
- **Availability:** failure of one host does not bring down the whole platform, because workloads can fail over to other hosts.

Type-2 hypervisors remain useful for developers, testers and training labs where convenience matters more than performance and isolation. For the production datacenter in this scenario, however, they are unsuitable. The trade-off of choosing Type-1 is higher initial cost and the

need for skilled administrators and compatible hardware, but these are small compared with the cost of an outage or a breach in a mission-critical system.

1.5 Conclusion

Type-1 hypervisors outperform Type-2 hypervisors on every criterion that matters for mission-critical hosting: performance, security and manageability. They should therefore be adopted as the standard virtualization layer of the datacenter, combined with good practices such as timely patching, least-privilege administration and a protected management network.

Question 2: Workload Isolation for Finance, Healthcare and Student Portals

2.1 Problem Interpretation

A cloud provider wants to run three very different workloads on the same physical hardware: a **finance portal** (payment and account data), a **healthcare portal** (patient records) and a **student portal** (academic and personal records). These tenants have different sensitivity levels and are subject to different compliance rules (for example PCI DSS for card data and HIPAA-style rules for health data). The task is to analyze how the virtualization architecture keeps these workloads isolated from each other, and to identify two major attack surfaces that threaten that isolation.

2.2 How Virtualization Architecture Supports Isolation

Isolation in a virtualized datacenter is not provided by one single feature. It is created by **several layers** that work together, from the processor up to the management plane.

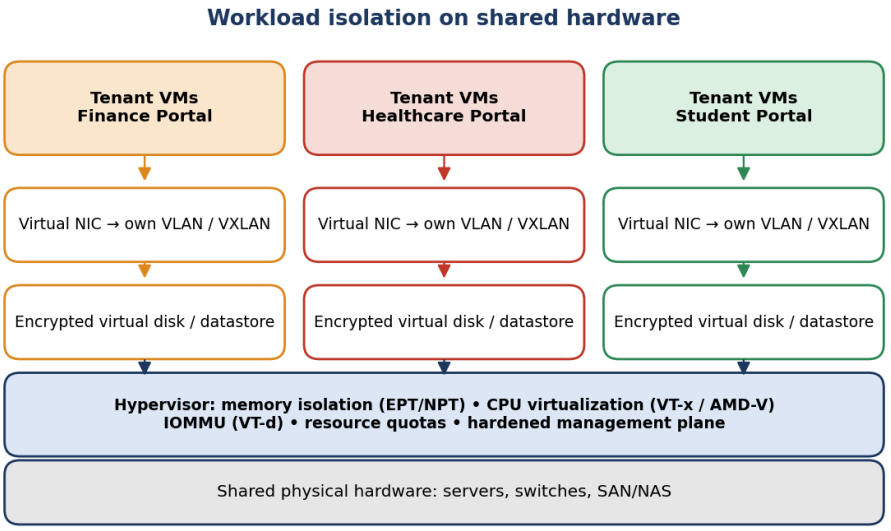


Figure 2.1: Layered isolation of three tenants on shared hardware

Layer	Isolation mechanism	Benefit for the three portals
CPU and memory	Hardware-assisted virtualization (VT-x / AMD-V) and nested page tables (EPT / NPT); each VM gets its own address space	One VM cannot read or write the memory of another VM, for example the finance VM cannot read healthcare data
I/O devices	IOMMU (VT-d) restricts DMA; virtual devices are emulated per VM	A device assigned to one VM cannot access memory of another

Layer	Isolation mechanism	Benefit for the three portals
Network	Separate virtual NICs, VLAN / VXLAN segments, virtual switches, distributed firewalls and micro-segmentation	Traffic of each portal is separated; only the explicitly allowed flows are permitted
Storage	Separate datastores or LUNs, per-tenant encryption keys, access control lists	A leaked disk or snapshot of one tenant is unreadable to others
Resources	CPU and memory reservations, limits and shares; storage and network QoS	Prevents the "noisy neighbour" problem, so a busy student portal cannot starve the finance portal
Placement	Affinity / anti-affinity rules, dedicated hosts or clusters for regulated tenants	Healthcare and finance VMs can be kept on separate hosts to meet compliance
Management	Role-based access control, separate admin domains, audit logs, MFA	Administrators of one tenant cannot manage VMs of another

Together these layers create what is called **multi-tenancy with strong isolation**: the tenants share hardware to reduce cost, but each one behaves as if it had its own private machine. For the highest-risk tenant (for instance healthcare), the provider can strengthen the design with dedicated hosts, encrypted VMs and confidential-computing hardware.

2.3 Two Major Attack Surfaces

Every layer that is shared between tenants is a potential attack surface. The two most important ones in this scenario are described below.

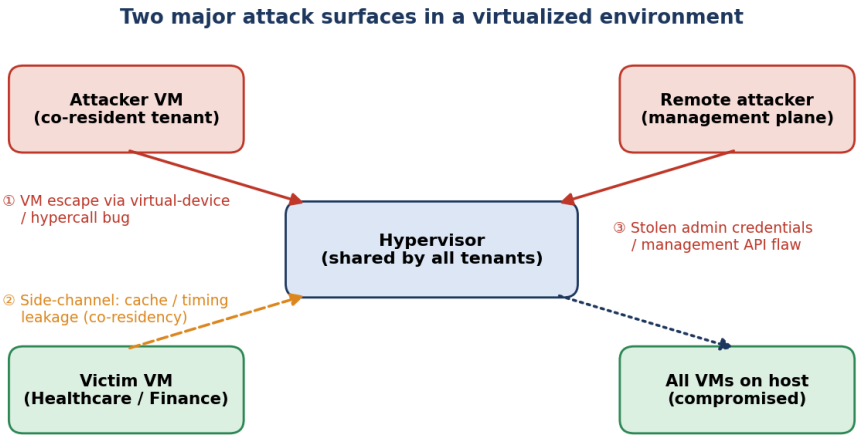


Figure 2.2: Attack paths against the shared hypervisor

Attack surface 1: The hypervisor and its virtual devices (VM escape)

The hypervisor is the single piece of software that every tenant depends on. It exposes interfaces to guests, such as emulated devices (virtual network card, disk controller, USB, graphics), hypercalls and shared-memory channels. A bug in one of these interfaces can

allow malicious code inside a guest VM to break out and run on the host, an attack known as **VM escape**. Once the attacker controls the hypervisor, the finance, healthcare and student VMs are all exposed. The same hypervisor is also reached through the **management plane** (management consoles and APIs); stolen administrator credentials give an attacker equivalent power, so this path is treated as part of the same attack surface.

Attack surface 2: Shared hardware resources (side-channel and co-residency attacks)
Even when memory isolation is perfect, tenants still share physical CPU caches, memory buses and execution units. An attacker who manages to place a VM on the same host as the victim (co-residency) can measure timing differences in the shared cache and infer secrets such as encryption keys. Speculative-execution flaws such as Spectre and Meltdown and memory-disturbance attacks such as Rowhammer belong to the same family. These attacks bypass software isolation, because no software boundary is formally crossed.

Attack surface	Example of attack	Impact on the three portals	Main countermeasures
1. Hypervisor / virtual devices / management plane	VM escape through a flawed virtual device; theft of admin credentials	Full loss of isolation; all tenants' data exposed	Prompt patching, minimal device emulation, secure boot, isolated management network, MFA and RBAC, logging and monitoring
2. Shared hardware (side channels)	Cache-timing attack from a co-resident VM; speculative-execution leakage	Leakage of keys and sensitive data from finance or healthcare VM	Dedicated hosts for sensitive tenants, anti-affinity placement, CPU microcode updates, core scheduling / disabling SMT, confidential computing

2.4 Recommended Design for the Provider

- Place the **healthcare and finance portals on dedicated hosts or clusters** (or at least with anti-affinity from the student portal) to reduce co-residency risk.
- Use **separate VLAN / VXLAN segments and distributed firewall rules** for each portal and allow only required traffic (zero-trust micro-segmentation).
- **Encrypt** virtual disks, snapshots and backups with a separate key per tenant, managed by a key-management service.
- Keep the **management network isolated**, use multi-factor authentication and role-based access, and log all administrative actions.
- Apply hypervisor, firmware and CPU microcode **patches quickly**, and monitor for abnormal behaviour such as unusual cache activity or unexpected VM migrations.

2.5 Conclusion

Virtualization architecture provides isolation through hardware-assisted memory protection, controlled I/O, virtual networking, separate storage and strict management controls. The isolation is strong but not perfect: the shared hypervisor (VM escape) and the shared hardware (side channels) are the two main attack surfaces. A layered, defence-in-depth design with dedicated placement for regulated tenants keeps the risk at an acceptable level.

Question 3: Analysis of the VM Host Utilization Snapshot

3.1 Problem Interpretation

The monitoring snapshot of a VM host shows four values: **CPU 92%**, **memory 88%**, **storage I/O latency 35 ms** and **average VM boot time 170 seconds**. The task is to explain the likely bottlenecks and to recommend corrective actions. Since the snapshot is only a single point in time, the analysis below identifies the most probable causes and also states which extra measurements should be taken to confirm them.

3.2 Reading the Metrics

The table compares each observed value with commonly used rule-of-thumb limits. These limits differ slightly between vendors and workloads, so they should be treated as guidance and not as fixed laws.

Metric	Observed	Healthy guideline	Interpretation
CPU utilization	92%	Below about 70–80% sustained	Very little headroom. VMs probably wait in the run queue (high CPU ready time), so applications slow down even though the CPU looks “busy”.
Memory utilization	88%	Below about 80–85%	Host is close to memory exhaustion. The hypervisor may start ballooning, compressing or swapping VM memory to disk.
Storage I/O latency	35 ms	Below about 10–20 ms	Storage is slow or overloaded. Requests wait in the queue, which delays every VM that reads or writes disk.
Average VM boot time	170 s	Typically around 30–60 s	A symptom of the other three problems; booting many VMs at once is very I/O and CPU intensive.

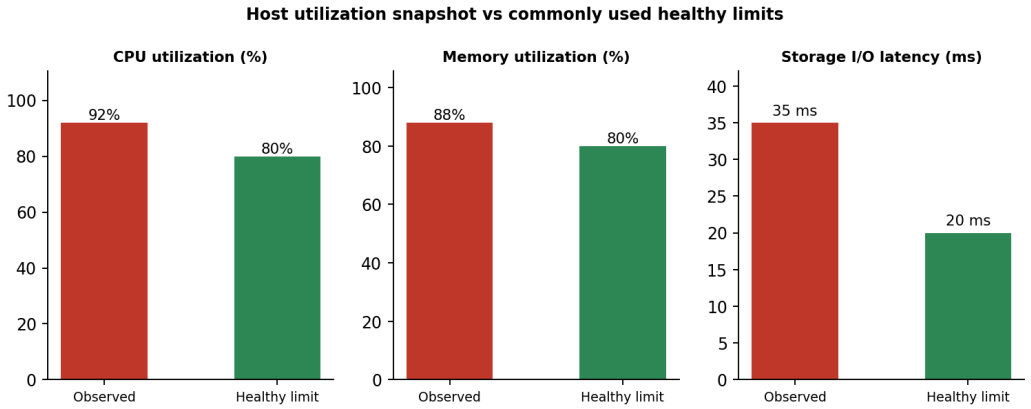


Figure 3.1: Observed values compared with healthy guidelines

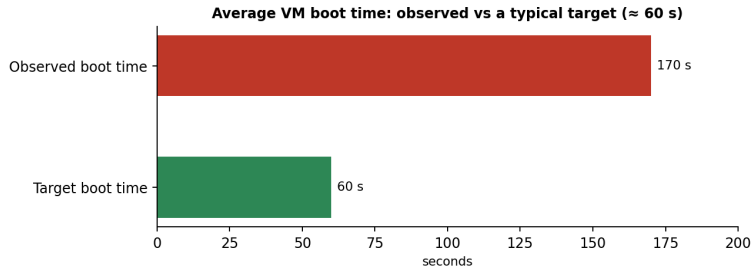


Figure 3.2: VM boot time compared with a typical target

3.3 Likely Bottlenecks

Bottleneck 1: CPU over-commitment

At 92% utilization the host has almost no spare processing capacity. When more virtual CPUs are scheduled than physical cores can serve, VMs spend time waiting for a core instead of running. This waiting time (CPU ready or steal time) increases response time and also slows down the boot sequence, because operating-system start-up is CPU-heavy. The likely cause is that too many VMs, or VMs with too many vCPUs, have been placed on this host.

Bottleneck 2: Memory pressure

Memory at 88% means that any additional load, such as several VMs booting together, can push the host past its limit. The hypervisor then uses reclamation techniques: first **ballooning** and **compression**, and finally **swapping** guest memory to disk. Swapping is the most harmful because it turns memory accesses, which take nanoseconds, into disk accesses, which take milliseconds. It also generates extra I/O on the storage system, which is already slow.

Bottleneck 3: Storage I/O latency (probably the dominant one for boot time)

A latency of 35 ms is high for primary storage. Booting a VM involves reading many small blocks of the operating-system image, so boot time is very sensitive to disk latency. Likely reasons are a shared datastore that is overloaded, spinning disks (HDD) instead of SSDs, a **boot storm** (many VMs starting together), swap traffic from memory pressure, or a congested storage network.

Combined effect

The three problems reinforce each other: memory pressure creates swap I/O, which raises storage latency, which makes VMs wait longer, which keeps CPU and memory occupied for longer. The result is the 170 s boot time. The root cause is therefore **over-subscription of the host together with insufficient storage performance**, not a single failing component.

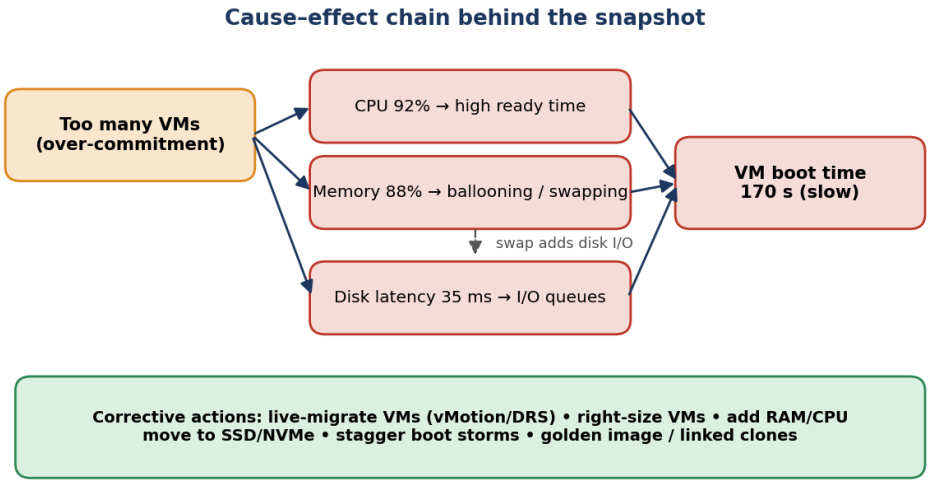


Figure 3.3: Cause–effect chain and corrective actions

3.4 Recommended Corrective Actions

Action	Bottleneck addressed	How it helps	Time frame
Live-migrate (vMotion / live migration) some VMs to less-loaded hosts; enable automatic load balancing (DRS)	CPU, memory	Brings host utilization down to about 70–75% without downtime	Immediate
Identify noisy or idle VMs and shut down, throttle or relocate them	CPU, memory, I/O	Frees resources quickly; shows which VMs cause the load	Immediate
Right-size VMs: reduce unnecessary vCPUs and RAM; remove memory over-commitment	CPU, memory	Reduces scheduling delay and reclamation	Short term
Stagger VM start-up and limit the number of simultaneous boots	I/O, CPU	Avoids boot storms	Short term
Use golden images with linked / thin clones and caching of boot data	Storage, boot time	Many VMs read the same cached blocks, so disk reads fall sharply	Short term
Move the datastore to SSD / NVMe or an all-flash tier; use multipathing and a dedicated storage network	Storage latency	Lowers latency to a few milliseconds	Medium term

Action	Bottleneck addressed	How it helps	Time frame
Add RAM and CPU capacity or add another host to the cluster	CPU, memory	Permanent capacity increase	Medium term
Set monitoring alerts and perform capacity planning (keep peak utilization below about 75–80%)	All	Prevents the problem from returning	Long term

How to verify that the actions worked

After the changes the following measurements should be tracked: CPU ready time (%), host memory usage and the rate of ballooning and swapping, datastore read and write latency (target below 10–20 ms), IOPS and queue depth, and the average boot time of a VM (target of roughly 60 s or less). Improvement in all four confirms that the root causes have been removed.

3.5 Conclusion

The snapshot shows a host that is overloaded on CPU and memory and that is slowed down further by storage latency. The most effective approach is to redistribute workloads immediately, right-size the VMs, reduce boot storms, and then invest in faster storage and extra capacity. With these measures utilization drops to a safe range and boot time returns to a normal level.

Question 4: Software Defined Datacenter vs Traditional Datacenter

4.1 Problem Interpretation

The question asks how a **Software Defined Datacenter (SDDC)** improves **agility** when compared with a traditional datacenter, and requires the answer to be supported by compute, storage and network virtualization considerations. In this context, agility means the speed and ease with which an organization can provision resources, scale them up or down, change configuration, and recover from failures.

4.2 The Traditional Datacenter

In a traditional datacenter the infrastructure is organized in **silos**: each application or department often has its own servers, storage arrays and network devices. Configuration is mostly manual and hardware-specific, and a new request usually follows a ticket-based process (server order, rack and cable, install, configure storage, configure VLAN and firewall). The result is slow provisioning (days or weeks), low average utilization because capacity is reserved for peak load, and a high risk of human error.

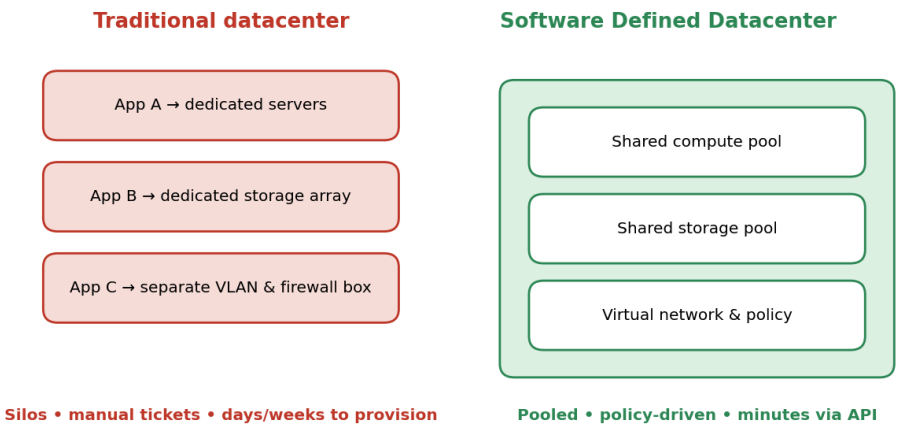


Figure 4.1: Siloed traditional datacenter and pooled SDDC

4.3 The SDDC Architecture

In an SDDC all infrastructure elements (compute, storage, networking and security) are **virtualized and delivered as software-defined services**. The underlying hardware is standardized and treated as a pool of resources. Control is moved from the hardware into software, so the entire datacenter can be configured, automated and monitored through policies and APIs from a central management and orchestration layer.

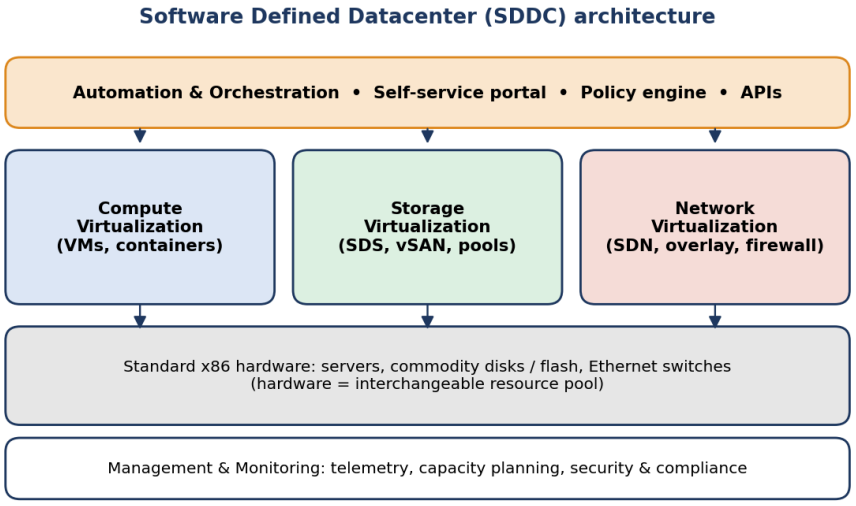


Figure 4.2: Layered SDDC architecture

4.4 Compute, Storage and Network Virtualization Considerations

Compute virtualization

Hypervisors (and containers) abstract CPU and memory from the physical servers. Because a VM is just a software object, it can be created from a template in minutes, resized without opening the server, moved between hosts with live migration, and restarted automatically on another host after a failure. Server consolidation raises utilization from typically low levels to much higher levels, which saves hardware, space and energy. **Considerations:** capacity planning and avoiding over-commitment, licensing, and the performance overhead of virtualization for very latency-sensitive workloads.

Storage virtualization

Software-defined storage pools the disks of many servers (for example hyper-converged storage) and presents them as logical volumes with policy-based features such as replication, thin provisioning, snapshots and tiering. A new volume can be provisioned by an API call instead of by configuring a physical array, and capacity can be expanded by adding nodes. **Considerations:** performance isolation between tenants, data protection policies, network bandwidth for replication and rebuilds, and the migration of legacy data.

Network virtualization

Software-defined networking (SDN) separates the control plane from the data plane, and overlay technologies such as VXLAN create virtual networks on top of the physical network. Virtual switches, routers, load balancers and distributed firewalls are created and configured in software, and the security policy follows the VM when it moves. A new isolated network for an application can therefore be created in seconds, and micro-segmentation can be applied without re-cabling. **Considerations:** overlay and underlay troubleshooting, controller availability, and the skills required for network automation.

4.5 How the SDDC Improves Agility

Aspect of agility	Traditional datacenter	Software Defined Datacenter
Provisioning time	Days to weeks; manual, ticket-based	Minutes; automated through templates, self-service portal and APIs
Scalability	Buy and install hardware in advance for peak demand	Elastic: add commodity nodes and pool them; scale applications up or down on demand
Resource utilization	Low; resources are tied to individual applications	High; shared pools and dynamic placement
Change management	Manual and device-by-device; error-prone	Policy-based, consistent, repeatable (infrastructure as code)
Availability and recovery	Hardware-specific clustering and slow DR	Live migration, automatic restart, replication and quick DR orchestration
Security	Perimeter firewalls; hardware-bound policy	Micro-segmentation and policies that follow the workload
Cost model	High capital expenditure; specialized hardware	Commodity hardware; pay-as-you-grow; lower operating expenditure
Operations	Separate teams for server, storage and network	Unified management, monitoring and automation

The common factor behind all of these improvements is **abstraction and automation**. Because infrastructure is described in software, it can be copied, versioned, tested and deployed repeatedly. Development teams can obtain a complete environment (servers, storage and network) in minutes, which supports DevOps practices, faster releases and quick response to changing business demand.

4.6 Challenges to Consider

- Initial investment in virtualization software, automation tools and staff training.
- Added complexity of abstraction layers (overlays, controllers) that need good monitoring.
- Performance overhead for specialized workloads; some may need bare-metal or hardware acceleration.
- Security and governance: the management and orchestration layers become high-value targets and must be protected.

4.7 Conclusion

An SDDC improves agility by pooling hardware, virtualizing compute, storage and network, and controlling everything through software and policy. Compared with a traditional

datacenter it provisions resources in minutes rather than weeks, scales elastically, uses hardware more efficiently and makes change safer. The challenges are real but manageable with planning, skills and strong management-layer security.

Question 5: Identity Mismatch in a Multi-Cloud Federation

5.1 Problem Interpretation

A multi-cloud federation connects several cloud providers so that users and applications can work across them. In the given project the federation fails because the **identities used by the providers do not match**. The task is to analyze why this happens and to propose a secure and privacy-preserving identity design that uses federation concepts.

5.2 Analysis of the Problem

Each provider maintains its own identity system with its own user identifiers, attribute names, role models and authentication protocols. When these systems are joined without a common trust and mapping layer, the same person appears as several unrelated identities. This leads to failed logins, wrong or excessive privileges, duplicate accounts and the impossibility of auditing a user's actions across clouds.

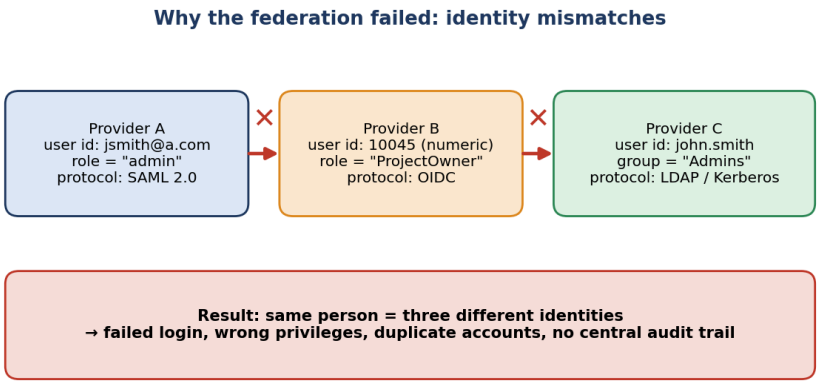


Figure 5.1: Same user, three incompatible identities

Root cause	Example in the project	Consequence
Different identifiers	jsmith@a.com in Provider A, numeric ID 10045 in Provider B, john.smith in Provider C	User cannot be matched; duplicate accounts
Different attribute and role schemas	"admin", "ProjectOwner" and "Admins" all mean something similar but are not equal	Wrong permissions: too few (access denied) or too many (privilege escalation)
Different protocols	SAML 2.0, OpenID Connect and LDAP / Kerberos used by different providers	Providers cannot understand each other's tokens
No common trust framework	Certificates and metadata exchanged manually or not at all	Tokens cannot be validated; trust is not verifiable
No lifecycle synchronization	User removed in one cloud but active in the others	Orphaned accounts and security risk

Root cause	Example in the project	Consequence
Uncontrolled attribute sharing	Full user profile sent to every provider	Privacy violation and unnecessary data exposure

5.3 Federation Concepts Used in the Solution

- **Identity Provider (IdP):** the system that authenticates the user and issues a signed statement (assertion or token) about the user.
- **Service Provider (SP) / Relying Party:** the cloud service that trusts the IdP's statement instead of keeping its own password for the user.
- **Trust relationship:** established by exchanging metadata and public keys so that signatures can be verified.
- **Single Sign-On (SSO):** the user authenticates once and accesses services in several clouds.
- **Standard protocols:** SAML 2.0 and OpenID Connect (built on OAuth 2.0) for authentication and authorization; SCIM for automated user provisioning and de-provisioning.
- **Claims / attributes:** pieces of information about the user (role, tenant, e-mail) carried inside the token.
- **Identity broker (federation hub):** a central component that translates between protocols and maps attributes, so that each provider needs only one trust connection.

5.4 Proposed Secure Identity and Privacy Design

The proposed design uses a **hub-and-spoke federation**. The organization's home IdP (Provider A) authenticates users. A federation hub / identity broker sits between the IdP and all other providers, which act as service providers. The hub performs protocol translation and attribute mapping, so providers never need to know each other's internal identifier or role formats.

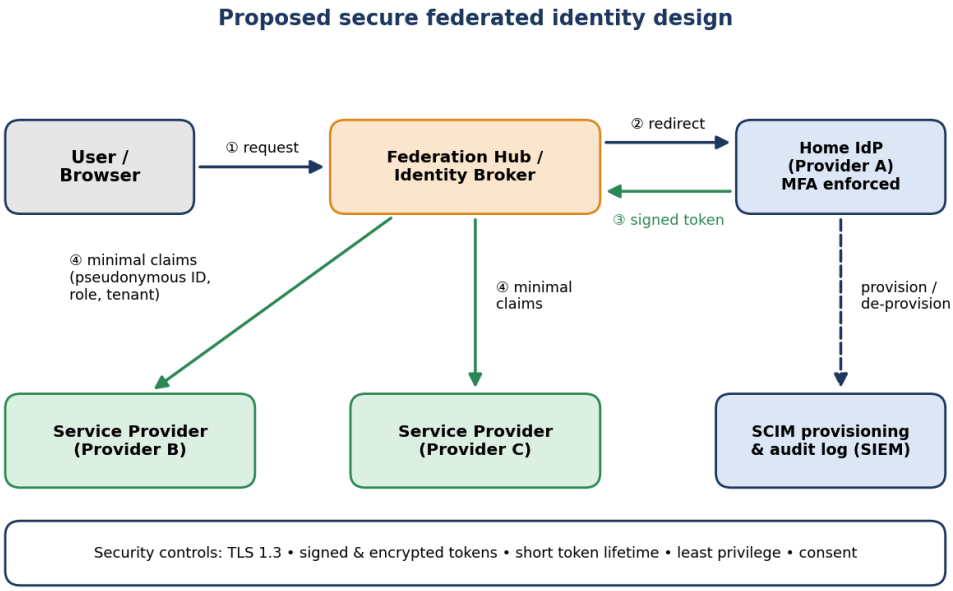


Figure 5.2: Hub-and-spoke federated identity design

Step-by-step flow

1. The user requests a service at Provider B or C.
2. The service redirects the user to the federation hub, which redirects the user to the home IdP.
3. The IdP authenticates the user with **multi-factor authentication** and returns a signed token or assertion.
4. The hub validates the signature, maps the attributes to the common schema, and issues a new token containing only the **minimum required claims**.
5. The service provider validates the token, creates a short-lived session and applies its own authorization policy.
6. User accounts are created and removed automatically in the providers through SCIM, and every event is sent to a central audit log (SIEM).

Identity design decisions

Component	Design decision
Common identity schema	Define a single canonical set of attributes (for example subject, tenant, role, group). The hub maps each provider's local names to this schema, so "admin", "ProjectOwner" and "Admins" map to one federated role.
Unique identifiers	Use one immutable federated identifier per user at the IdP; do not rely on e-mail or user names, which can change or be reused.
Protocols	Standardize on OpenID Connect or SAML 2.0 between the IdP, hub and providers; the hub translates for providers that use other protocols.

Component	Design decision
Authorization model	Role-based or attribute-based access control: roles are mapped centrally, and each provider enforces least privilege locally.
Lifecycle	SCIM or equivalent automation for joiner, mover and leaver events, so that a user disabled at the IdP loses access in all clouds quickly.

Privacy design decisions

- **Data minimization:** only the attributes needed by each service are released; a provider never receives the full profile.
- **Pseudonymous (pairwise) identifiers:** each service provider receives a different opaque identifier for the same user, so providers cannot link the user's activity across clouds.
- **Consent and purpose limitation:** attribute release is governed by policy and, where required, user consent; data is used only for the stated purpose.
- **Protection of data in transit and at rest:** TLS 1.2+ (preferably TLS 1.3) for all exchanges; tokens signed and, when they carry sensitive claims, encrypted.
- **Retention and auditability:** logs keep the minimum personal data needed, are access-controlled, and are retained according to policy and applicable law (such as the GDPR or India's Digital Personal Data Protection Act, 2023).

Security controls

Control	Implementation
Strong authentication	Multi-factor authentication at the IdP; step-up authentication for sensitive operations.
Token security	Digital signatures (for example RS256 / ECDSA), checks of issuer, audience and expiry, nonce / replay protection, short token lifetimes and refresh-token rotation.
Key and metadata management	Automatic key rotation, signed federation metadata, certificate validation and revocation.
Network protection	Mutual TLS between hub and providers, isolated management network, API gateway with rate limiting.
Monitoring and response	Central logging of authentication events, anomaly detection, alerts and a documented incident-response and revocation procedure.
Time synchronization	NTP on all components, because token validity checks depend on correct clocks.

5.5 Implementation Roadmap

Phase	Activities
1. Assess	Inventory identity stores, protocols, attributes and roles of every provider; identify duplicates and orphan accounts.
2. Design	Define the canonical schema, role mapping, trust framework and privacy policy.
3. Build	Deploy the identity broker, configure metadata and keys, enable MFA and SCIM.
4. Pilot	Federate one application per provider; test login, role mapping, logout and de-provisioning.
5. Roll out and monitor	Migrate remaining services in waves; monitor logs and review access regularly.

5.6 Conclusion

The failure of the federation was caused by inconsistent identifiers, attributes, protocols and trust arrangements. A federated design based on an IdP, an identity broker, standard protocols, strong authentication and privacy-by-design principles provides one federated identity that works across all providers, with consistent access rights, automatic account removal, limited personal data and full auditability. This resolves the identity mismatch and gives secure single sign-on across clouds without exposing more personal data than necessary.

Overall Conclusion and References

Overall Conclusion

This assessment analyzed five aspects of virtualization in cloud computing. Type-1 hypervisors were found to be the correct choice for mission-critical datacenters because of their performance, security and manageability. Workload isolation relies on layered controls, but the hypervisor and shared hardware remain the main attack surfaces. The utilization snapshot revealed a combined CPU, memory and storage bottleneck that can be corrected by rebalancing, right-sizing and faster storage. The Software Defined Datacenter provides much greater agility than the traditional datacenter through virtualization of compute, storage and network. Finally, a federated identity design with an identity broker, standard protocols and privacy controls solves the identity mismatch in multi-cloud environments.

References

[1] Popek, G. J. and Goldberg, R. P. (1974). Formal requirements for virtualizable third generation architectures. Communications of the ACM, 17(7).

[2] NIST Special Publication 800-125: Guide to Security for Full Virtualization Technologies.

[3] NIST Special Publication 800-145: The NIST Definition of Cloud Computing.

[4] NIST Special Publication 800-63C: Digital Identity Guidelines – Federation and Assertions.

[5] OASIS. Security Assertion Markup Language (SAML) V2.0 Technical Overview.

[6] OpenID Foundation. OpenID Connect Core 1.0; IETF RFC 6749: The OAuth 2.0 Authorization Framework.

[7] IETF RFC 7644: System for Cross-domain Identity Management (SCIM) Protocol.

[8] Course material, CSA15 Cloud Computing and Big Data Analytics, SIMATS Engineering.